

Procedure

Step 1 — Initialize the Project

Create the project, install dependencies, and set up the folder structure.

```
mkdir welcm-h4cker && cd welcm-h4cker
npm init -y
npm install express compression
```

Project structure: server.js (Express server), public/ folder containing index.html, terminal.html, stylesheets, scripts, and data/virtual_fs.json (virtual filesystem data).

Step 2 — Create the Virtual Filesystem

Create public/data/virtual_fs.json — a JSON structure mimicking a Linux filesystem with directories (home, etc, var, opt), readable files, and a hidden flag file (flag.txt) with property "hidden": true placed inside /home/user/admin/secret/.

Step 3 — Build the Secure Express Server (server.js)

The server implements both functional and nonfunctional security:

- a) Middleware — Nonfunctional Security:
 - Enable gzip compression using compression() middleware.
 - Serve static files with ETag, Last-Modified, and max-age=1y in production.
- b) Filesystem Sanitization — Functional Security:
 - Load virtual_fs.json once at startup (in-memory caching).
 - Create getSanitizedFS() that deep-clones the filesystem and strips all content from hidden files before sending to clients via the /api/filesystem endpoint.
- c) Secure Terminal API — Core Functional Security:

POST /api/terminal/execute applies the following security layers:

Security Measure	Implementation
Input Validation	Check command is a non-empty string; args is an array; cwd starts with "/"

Command Whitelisting	Only allow: ls, cd, cat, pwd, help, clear, find, grep, history, whoami
Argument Sanitization	Regex strips dangerous characters; each arg truncated to 256 chars
Path Traversal Prevention	resolvePath() handles ".." safely — cannot escape virtual root "/"
No Shell Execution	All commands are pure JS switch/case logic on in-memory JSON — no eval(), no child_process

d) Cache-Control Headers — Nonfunctional Security:

- Static pages: Cache-Control: public, max-age=3600
- Terminal execute / health: Cache-Control: no-store
- Stats API: Cache-Control: public, max-age=10

Step 4 — Build the Frontend

- index.html — Landing page with matrix rain canvas, system metrics dashboard (/api/stats), hacker quote section (/api/quote), and a link to the terminal.
- terminal.html + terminal.js — Full-screen terminal UI. Every command is sent to the server via POST /api/terminal/execute. User input is escaped with escapeHtml() (DOM-based XSS prevention) before rendering in the terminal output.

Step 5 — Run and Test

npm start

Open <http://localhost:3000> and <http://localhost:3000/terminal>

Test	Command / Action	Expected Result
Whitelisted command	ls	Lists directory contents
Blocked command	rm -rf /	rm: command not found (400)

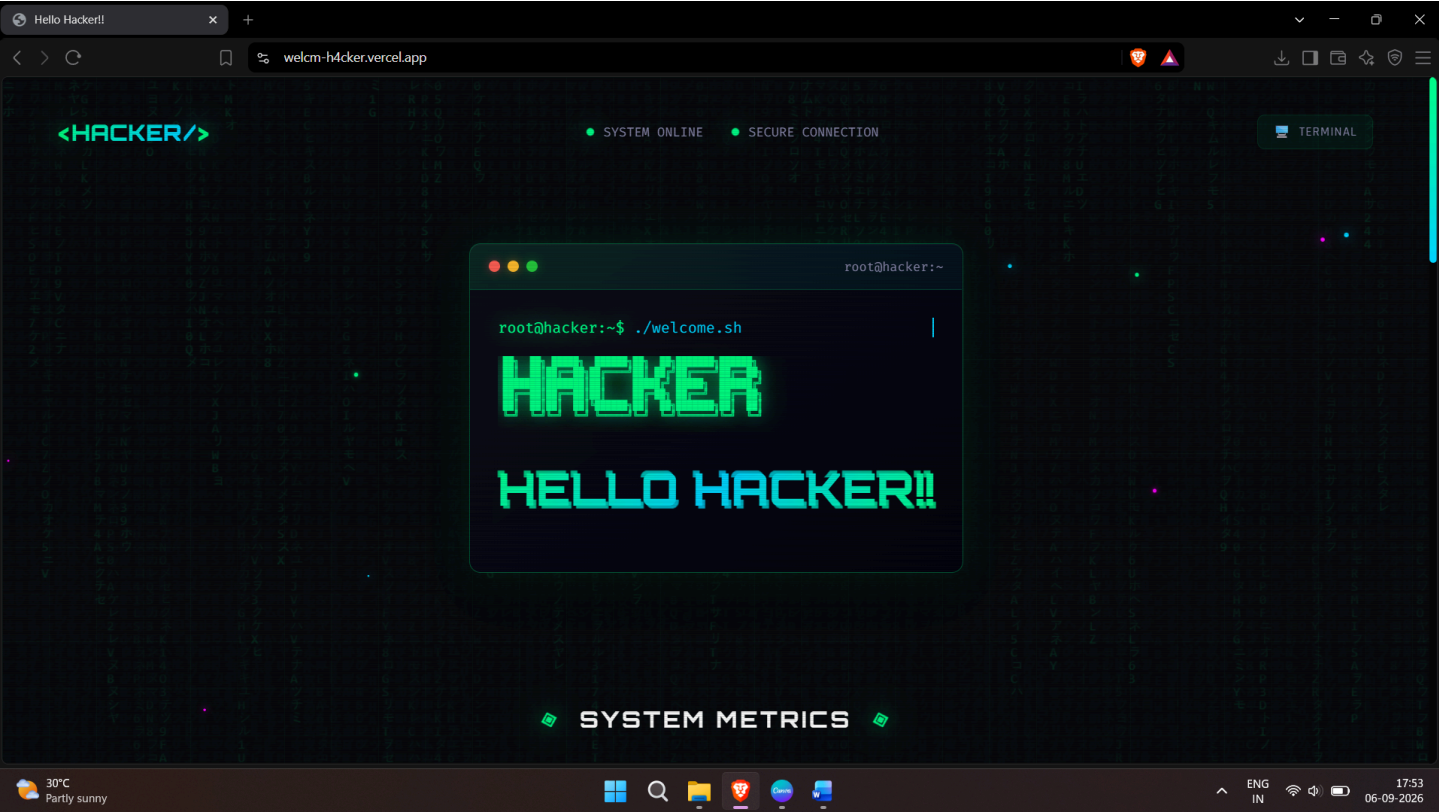
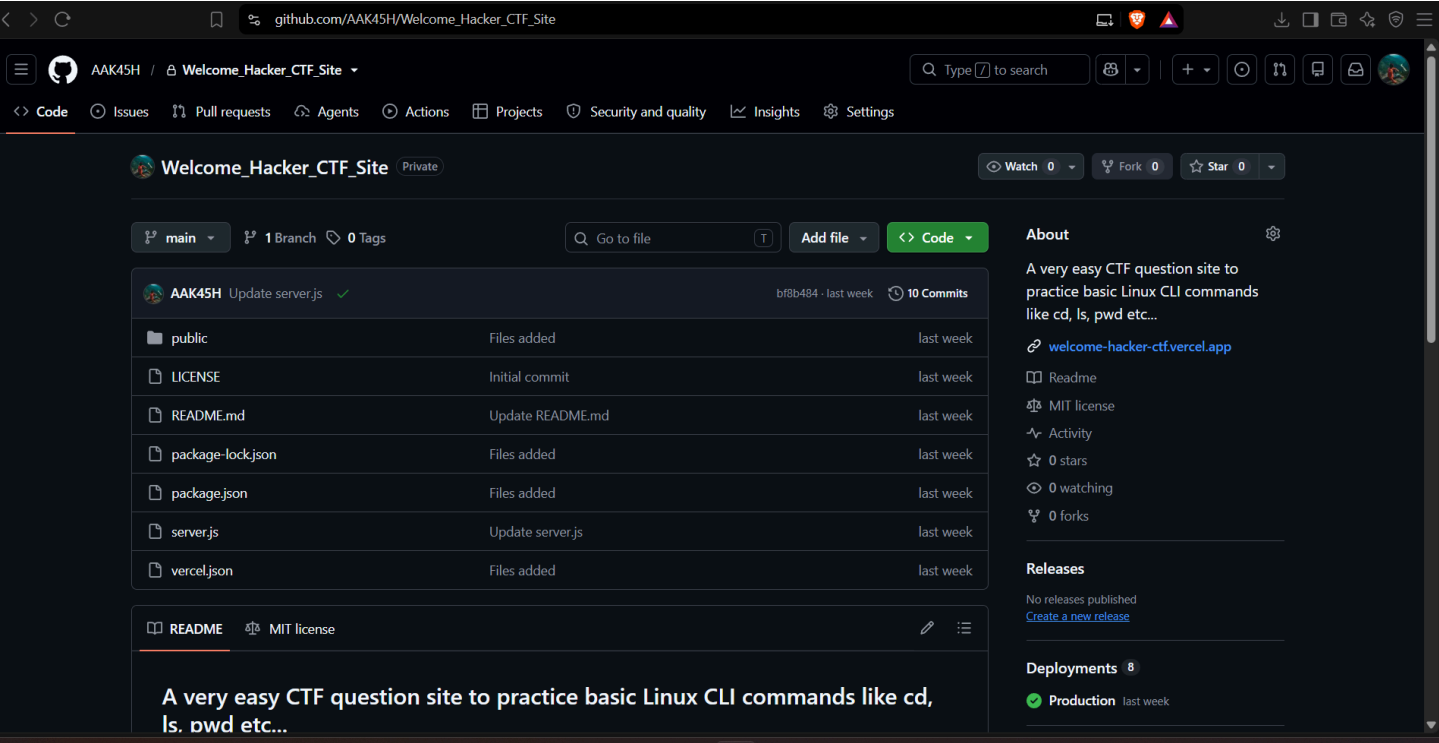
Path traversal	cat ../../../../etc/passwd	Resolves within virtual FS only
Hidden file protection	ls in /home/user/admin/secret	Shows key.log only (flag hidden)
Reveal hidden file	ls -a in secret dir	Shows flag.txt
Read flag	cat flag.txt	FL4G{7H3_53CR37_15_4LW4Y5_51MPL3}
API sanitization	GET /api/filesystem	Flag content stripped from response
Health check	GET /health	{ "status": "ok" }

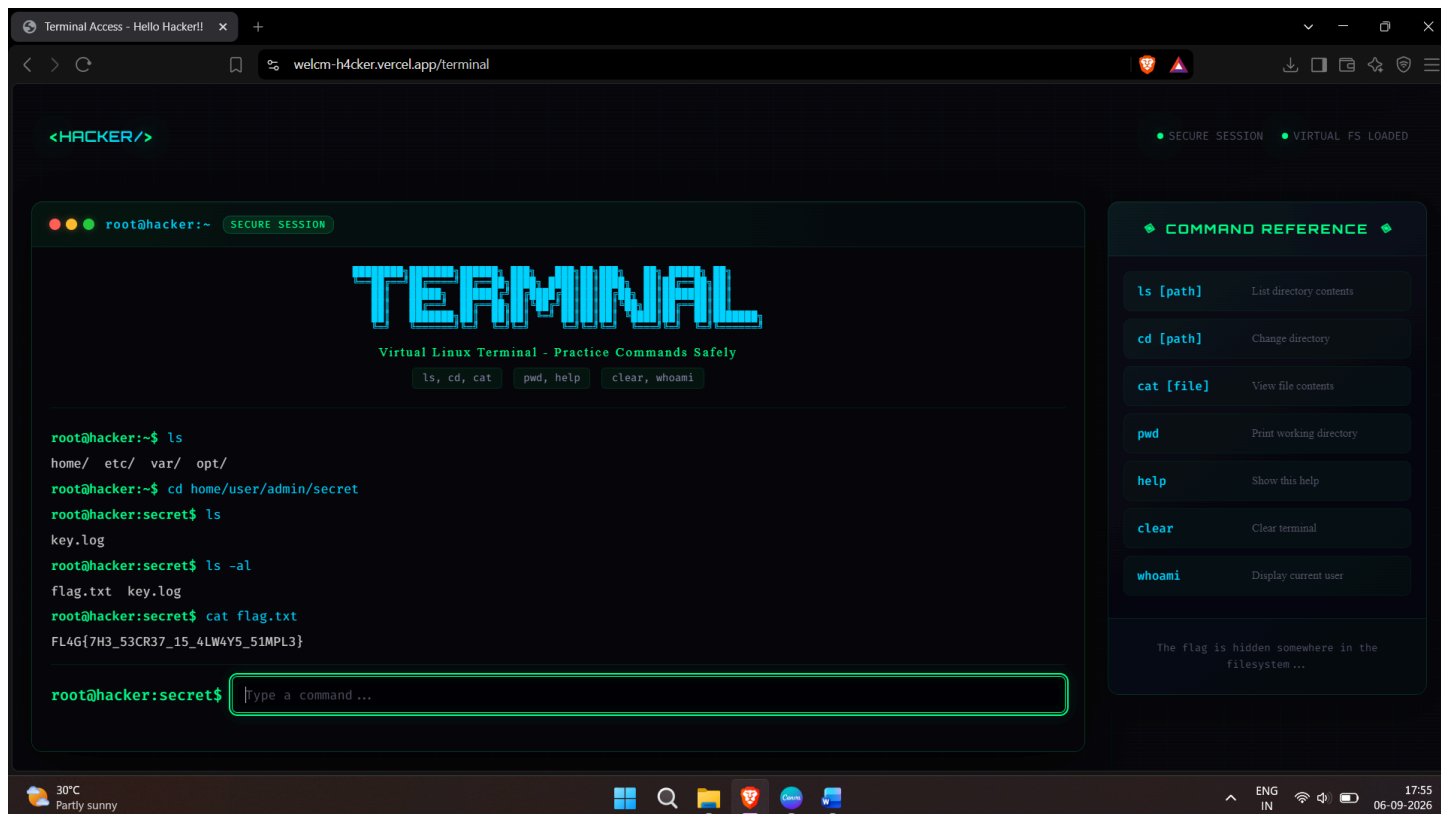
Application Architecture

welcm-h4cker/

— server.js	← Express server (API + routing + security logic)
— package.json	← Node.js project manifest
— vercel.json	← Vercel serverless deployment configuration
— public/	← Static frontend assets
— index.html	← Landing page (hacker dashboard)
— styles.css	← Landing page styles (matrix, glitch, glass effects)
— script.js	← Landing page JavaScript (matrix rain, stats, quotes)
— terminal.html	← Virtual terminal page
— terminal.css	← Terminal page styles
— terminal.js	← Terminal UI logic (VirtualFileSystem & TerminalUI classes)
— data/	
— virtual_fs.json	← Virtual filesystem data (directory tree + file contents)

Output Screenshots





Github link: https://github.com/AAK45H/Welcome_Hacker_CTF_Site

Site Link: <https://welcm-h4cker.vercel.app>